Mini-project: expense tracker

🎯 Today's goal

Today there's almost nothing new. Instead you'll build one small, real program step by step. By the end you will:

- store expenses as a **list of dictionaries**, like rows in a table
- write functions that **add** an expense and **refuse** bad ones
- calculate totals, totals per category and the biggest expense
- pick the expenses of one month and check a budget
- **save** everything to a CSV file, **load** it back, and print a neat report

🤔 Why this matters

Knowing the words of a language is not the same as speaking it. A project shows you how the pieces from the first week fit together: lists, dictionaries, `if`, loops, functions, files and errors.

It's also the same job a data analyst does every day, just smaller: **collect** records, **check** them, **summarise** them by group, and **report** the result. From Day 10, pandas will do much of this in one line, and you'll understand what it's doing for you.

📋 Step 1: the data

Each expense is a **dictionary** with four keys: `date`, `category`, `amount` and `note`. All expenses together are a **list** of these dictionaries. Think of each dictionary as one row of a spreadsheet, and the keys as the column names. Dates are written as text in the form `"year-month-day"`, like `"2026-03-02"`.

```
# Made-up expenses of a student for March and April 2026
expenses = [
    {"date": "2026-03-02", "category": "food", "amount": 8.50, "note": "lunch"},
    {"date": "2026-03-05", "category": "travel", "amount": 2.40, "note": "bus"},
    {"date": "2026-03-10", "category": "books", "amount": 24.99, "note": "textbook"},
    {"date": "2026-03-15", "category": "fun", "amount": 12.00, "note": "cinema"},
    {"date": "2026-03-28", "category": "food", "amount": 31.75, "note": "groceries"},
    {"date": "2026-04-01", "category": "rent", "amount": 350.00, "note": "room"},
    {"date": "2026-04-03", "category": "food", "amount": 9.20, "note": "pizza"},
]

print("Number of expenses:", len(expenses))
print("The first one:", expenses[0])
print("Its amount:", expenses[0]["amount"])
```

```
Number of expenses: 7
The first one: {'date': '2026-03-02', 'category': 'food', 'amount': 8.5,
'note': 'lunch'}
Its amount: 8.5
```

➕ Step 2: adding an expense (and refusing bad ones)

A function `add_expense()` builds the dictionary and appends it to the list. Before that, it **validates** the data, which means it checks that the data makes sense. An amount of zero or less, or an unknown category, is refused with a `ValueError` (exactly the `raise` trick from yesterday). Checking data when it arrives saves you from strange results later.

```
CATEGORIES = ["food", "travel", "rent", "fun", "books"]

def add_expense(expenses, date, category, amount, note=""):
    """Check one expense, then add it to the list."""
    if amount <= 0:
        raise ValueError(f"The amount must be more than 0, got {amount}")
    if category not in CATEGORIES:
        raise ValueError(f"Unknown category '{category}'. Use one of {CATEGORIES}")
    expenses.append({"date": date, "category": category,
                     "amount": amount, "note": note})

add_expense(expenses, "2026-04-04", "travel", 2.40, "bus")
print("Now there are", len(expenses), "expenses.")
```

```
Now there are 8 expenses.
```

Careful: every time you run this box, the bus ticket is added *again*. If your numbers look too big, run the boxes again from Step 1, which starts the list fresh.

Writing the category list in CAPITAL letters is a Python habit that means "this is a fixed setting, don't change it while the program runs". Now let's try a bad expense, on purpose:

💡 This example has a mistake on purpose. Run it and read the error.

```
add_expense(expenses, "2026-04-05", "food", -4.00, "refund?")
```

> ⚠️ **ValueError**
>
> The kind of value is right but the value itself isn't, like int("hello").
>
> ```
> Traceback (most recent call last):
>   line 1, in <module>
>     add_expense(expenses, "2026-04-05", "food", -4.00, "refund?")
>     ~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
>   line 6, in add_expense
> ValueError: The amount must be more than 0, got -4.0
> ```

Good: the error message says exactly what's wrong. When many expenses arrive at once, we don't want one bad entry to stop everything, so we catch the error with `try` / `except`:

```
# Made-up new entries: (date, category, amount, note)
new_entries = [
    ("2026-04-06", "fun", 6.50, "museum"),
    ("2026-04-07", "pets", 15.00, "cat food"),
    ("2026-04-08", "fun", 0, "free concert"),
]

for date, category, amount, note in new_entries:
    try:
        add_expense(expenses, date, category, amount, note)
        print("Added:", note)
    except ValueError as err:
        print("Refused:", err)

print("Now there are", len(expenses), "expenses.")
```

```
Added: museum
Refused: Unknown category 'pets'. Use one of ['food', 'travel', 'rent', 'fun',
'books']
Refused: The amount must be more than 0, got 0
Now there are 9 expenses.
```

🧮 Step 3: totals

The first question everyone asks: *how much did I spend?* A loop adds up the `"amount"` of every expense:

```
def total(expenses):
    """Add up the amounts of all expenses."""
    result = 0
    for expense in expenses:
        result += expense["amount"]
    return result

print(f"Total spent: {total(expenses):.2f}")
```

```
Total spent: 447.74
```

The next question: *where did the money go?* We build a dictionary with one total per category. `totals.get(category, 0)` gives the total so far, or `0` the first time we meet a category. This "group and add up" pattern is everywhere in data analysis (on Day 11, pandas calls it `groupby`):

```
def totals_by_category(expenses):
    """Return a dict like {"food": 49.45, "rent": 350.0, ...}."""
    totals = {}
    for expense in expenses:
        category = expense["category"]
        totals[category] = totals.get(category, 0) + expense["amount"]
    return totals

for category, amount in totals_by_category(expenses).items():
    print(f"{category}: {amount:.2f}")
```

```
food: 49.45
travel: 4.80
books: 24.99
fun: 18.50
rent: 350.00
```

And the biggest single expense. First the long way with a loop, then a shortcut using `max()` with a `key`, like `sorted(..., key=...)` on Day 6:

```
biggest = expenses[0]
for expense in expenses:
    if expense["amount"] > biggest["amount"]:
        biggest = expense
print("Loop:    ", biggest["note"], biggest["amount"])

# Shortcut: "the max of the list, comparing by amount"
biggest = max(expenses, key=lambda e: e["amount"])
print("Shortcut:", biggest["note"], biggest["amount"])
```

```
Loop:     room 350.0
Shortcut: room 350.0
```

📅 Step 4: one month and a budget

Because our dates are text like `"2026-03-15"`, every March expense starts with `"2026-03"`. The string method `.startswith()` answers exactly that question with `True` or `False`:

```
print("2026-03-15".startswith("2026-03"))
print("2026-04-01".startswith("2026-03"))

def in_month(expenses, month):
    """Return only the expenses whose date starts with month, e.g. "2026-03"."""
    return [e for e in expenses if e["date"].startswith(month)]

march = in_month(expenses, "2026-03")
print(len(march), "expenses in March")
print(f"March total: {total(march):.2f}")
```

```
True
False
5 expenses in March
March total: 79.64
```

Notice how the functions work together: `total()` doesn't care whether it gets all expenses or only March. That's the power of small functions. Now a budget check with `if`:

```
def budget_message(spent, budget):
    """Say whether we stayed within the budget."""
    if spent > budget:
        return f"Over budget by {spent - budget:.2f}!"
    return f"OK, {budget - spent:.2f} left."

print("March:", budget_message(total(march), 100))
print("April:", budget_message(total(in_month(expenses, "2026-04")), 300))
```

```
March: OK, 20.36 left.
April: Over budget by 68.10!
```

💾 Step 5: save and load

A tracker that forgets everything isn't much use. We save the list to a CSV file: first a header row with the column names, then one row per expense.

```
import csv

COLUMNS = ["date", "category", "amount", "note"]

def save_expenses(expenses, file_name):
    """Write all expenses to a CSV file."""
    with open(file_name, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(COLUMNS)
        for e in expenses:
            writer.writerow([e["date"], e["category"], e["amount"], e["note"]])

save_expenses(expenses, "expenses.csv")

with open("expenses.csv") as f:
    print(f.read())
```

```
date,category,amount,note
2026-03-02,food,8.5,lunch
2026-03-05,travel,2.4,bus
2026-03-10,books,24.99,textbook
2026-03-15,fun,12.0,cinema
2026-03-28,food,31.75,groceries
2026-04-01,rent,350.0,room
2026-04-03,food,9.2,pizza
2026-04-04,travel,2.4,bus
2026-04-06,fun,6.5,museum
```

Loading is the reverse. Two things to remember from yesterday: the amount comes back as **text**, so we convert it with `float()`; and the file might not exist yet (the very first time you use the tracker), so we catch `FileNotFoundError` and start with an empty list:

```
def load_expenses(file_name):
    """Read expenses from a CSV file. Return an empty list if there is no file yet."""
    loaded = []
    try:
        with open(file_name, newline="") as f:
            reader = csv.reader(f)
            next(reader)                    # skip the header row
            for date, category, amount, note in reader:
                loaded.append({"date": date, "category": category,
                               "amount": float(amount), "note": note})
    except FileNotFoundError:
        print(f"No file called {file_name} yet, starting empty.")
    return loaded

loaded = load_expenses("expenses.csv")
print(len(loaded), "expenses loaded. Same total?", total(loaded) == total(expenses))

nothing = load_expenses("old_expenses.csv")
print(nothing)
```

```
9 expenses loaded. Same total? True
No file called old_expenses.csv yet, starting empty.
[]
```

🖨️ Step 6: a neat report

Finally, a report with the numbers lined up in columns. f-strings can set a **width** and an **alignment**:

Format	Means	Example result
`{text:<12}`	at least 12 characters wide, lined up on the **left**	`"food "`
`{amount:>8.2f}`	8 wide, lined up on the **right**, 2 decimals	`" 49.45"`

```
def print_report(expenses):
    print(f"{'Category':<12}{'Amount':>8}")
    print("-" * 20)
    for category, amount in totals_by_category(expenses).items():
        print(f"{category:<12}{amount:>8.2f}")
    print("-" * 20)
    print(f"{'TOTAL':<12}{total(expenses):>8.2f}")

print_report(loaded)
```

```
Category      Amount
--------------------
food           49.45
travel          4.80
books          24.99
fun            18.50
rent          350.00
--------------------
TOTAL         447.74
```

That's a complete little program: data in, checked, summarised, saved, loaded and reported. Try adding your own expenses in Step 2 and run everything again.

⚠️ Common mistakes

- Running the "add" box several times and adding the same expense again and again.
- Forgetting that values loaded from a CSV file are text: convert amounts with `float()`.
- A function that `print`s its result instead of `return`ing it: then you can't use the result in another function.
- Misspelling a dictionary key: `e["ammount"]` gives a `KeyError`.
- Checking data too late: validate it when it arrives, in one place (`add_expense`).

✅ Recap

- A list of dictionaries works like a table: one dictionary per row, keys as column names.
- Validate input and `raise ValueError` with a clear message; catch it with `try` / `except`.
- "Group and add up": `totals[key] = totals.get(key, 0) + value`.
- `text.startswith("2026-03")` picks out one month from dates written as text.
- Save with `csv.writer`, load with `csv.reader` + `float()` + `FileNotFoundError`.
- Line up columns with `{text:<12}` and `{number:>8.2f}`.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Add an expense

Write the function `add_expense(expenses, date, category, amount, note="")` yourself. It should:

- raise a `ValueError` if `amount` is 0 or less
- raise a `ValueError` if `category` is not in `CATEGORIES`
- otherwise append a dictionary with the keys `date`, `category`, `amount` and `note` to `expenses`

```
CATEGORIES = ["food", "travel", "rent", "fun", "books"]

def add_expense(expenses, date, category, amount, note=""):
    # 1. raise a ValueError if amount is 0 or less
    # 2. raise a ValueError if category is not in CATEGORIES
    # 3. append a dict with the keys date, category, amount and note
    pass
```

Hints:

1. `if amount <= 0:` then, indented, `raise ValueError("The amount must be more than 0")`.
2. `if category not in CATEGORIES:` then raise another `ValueError`.
3. `expenses.append({"date": date, "category": category, "amount": amount, "note": note})`

2. Totals per category

Write `category_totals(expenses)` that **returns** a dictionary with the total amount per category. For the list in the box it should return `{"food": 23.0, "travel": 5.0, "fun": 7.5}`.

```
expenses = [
    {"date": "2026-05-01", "category": "food", "amount": 10.0, "note": "lunch"},
    {"date": "2026-05-02", "category": "travel", "amount": 5.0, "note": "train"},
    {"date": "2026-05-02", "category": "food", "amount": 13.0, "note": "dinner"},
    {"date": "2026-05-04", "category": "fun", "amount": 7.5, "note": "bowling"},
]

def category_totals(expenses):
    totals = {}
    # Fill totals here
    return totals

print(category_totals(expenses))
```

Hints:

1. Loop over the expenses: `for e in expenses:`
2. Inside the loop: `totals[e["category"]] = totals.get(e["category"], 0) + e["amount"]`

3. Expenses of one month

Write `in_month(expenses, month)` that **returns** a new list with only the expenses whose date starts with `month` (for example `"2026-05"`), in the same order. Then store the May expenses in a variable called `may`.

```
expenses = [
    {"date": "2026-04-29", "category": "food", "amount": 6.0, "note": "snack"},
    {"date": "2026-05-01", "category": "food", "amount": 10.0, "note": "lunch"},
    {"date": "2026-05-15", "category": "fun", "amount": 7.5, "note": "bowling"},
    {"date": "2026-06-02", "category": "rent", "amount": 300.0, "note": "room"},
]

def in_month(expenses, month):
    # Return only the expenses of that month
    return []

may = in_month(expenses, "2026-05")
print(len(may), "expenses in May")
```

Hints:

1. `e["date"].startswith(month)` is `True` for the expenses you want.
2. Build a new list with a loop and `append`, or a list comprehension.
3. `return [e for e in expenses if e["date"].startswith(month)]`

4. Budget warning

Write `budget_warning(spent, budget)` that **returns** one of three messages (amounts with 2 decimals):

- if `spent` is more than `budget`: `"Over budget by 12.50"`
- else, if `spent` is at least 90% of `budget`: `"Careful: only 5.00 left"`
- otherwise: `"OK: 40.00 left"`

```
def budget_warning(spent, budget):
    # Return one of the three messages
    return ""

print(budget_warning(112.5, 100))
print(budget_warning(95, 100))
print(budget_warning(60, 100))
```

Hints:

1. Use `if`, `elif` and `else`. "At least 90%" is `spent >= 0.9 * budget`.
2. The money left is `budget - spent`; over budget by `spent - budget`.
3. `return f"Careful: only {budget - spent:.2f} left"`

5. Challenge: monthly report

Write `monthly_report(expenses, month)` that **returns** a dictionary with three keys about that month's expenses:

- `"count"`: how many expenses there are
- `"total"`: their total amount, rounded to 2 decimals
- `"top_category"`: the category with the biggest total

For May in the box it should return `{"count": 4, "total": 45.5, "top_category": "food"}`. You may write helper functions (like the ones from the lesson) in the same box.

```
expenses = [
    {"date": "2026-04-30", "category": "rent", "amount": 300.0, "note": "room"},
    {"date": "2026-05-01", "category": "food", "amount": 10.0, "note": "lunch"},
    {"date": "2026-05-02", "category": "fun", "amount": 18.0, "note": "concert"},
    {"date": "2026-05-09", "category": "food", "amount": 12.5, "note": "dinner"},
    {"date": "2026-05-20", "category": "travel", "amount": 5.0, "note": "train"},
]

def monthly_report(expenses, month):
    # 1. keep only this month's expenses
    # 2. count them and add up the amounts
    # 3. find the category with the biggest total
    return {}

print(monthly_report(expenses, "2026-05"))
```

Hints:

1. Start with `chosen = [e for e in expenses if e["date"].startswith(month)]`.
2. Build a dictionary of totals per category (like exercise 2), then find the key with the biggest value with a loop, or with `max(totals, key=totals.get)`.
3. Return `{"count": len(chosen), "total": round(total, 2), "top_category": top}`.

➡️ Tomorrow

NumPy: fast math on arrays. The Data Science phase begins! Today you added up numbers one by one with loops. Tomorrow you'll meet NumPy, a module that does maths on a whole list of numbers at once: double every price, find the hot days, or average each student's test scores in a single line.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Add an expense

```
CATEGORIES = ["food", "travel", "rent", "fun", "books"]

def add_expense(expenses, date, category, amount, note=""):
    if amount <= 0:
        raise ValueError(f"The amount must be more than 0, got {amount}")
    if category not in CATEGORIES:
        raise ValueError(f"Unknown category: {category}")
    expenses.append({"date": date, "category": category, "amount": amount,
"note": note})
```

2. Totals per category

```
expenses = [
    {"date": "2026-05-01", "category": "food", "amount": 10.0, "note": "lunch"},
    {"date": "2026-05-02", "category": "travel", "amount": 5.0, "note": "train"},
    {"date": "2026-05-02", "category": "food", "amount": 13.0, "note": "dinner"},
    {"date": "2026-05-04", "category": "fun", "amount": 7.5, "note": "bowling"},
]

def category_totals(expenses):
    totals = {}
    for e in expenses:
        totals[e["category"]] = totals.get(e["category"], 0) + e["amount"]
    return totals

print(category_totals(expenses))
```

3. Expenses of one month

```
expenses = [
    {"date": "2026-04-29", "category": "food", "amount": 6.0, "note": "snack"},
    {"date": "2026-05-01", "category": "food", "amount": 10.0, "note": "lunch"},
    {"date": "2026-05-15", "category": "fun", "amount": 7.5, "note": "bowling"},
    {"date": "2026-06-02", "category": "rent", "amount": 300.0, "note": "room"},
]

def in_month(expenses, month):
    return [e for e in expenses if e["date"].startswith(month)]

may = in_month(expenses, "2026-05")
print(len(may), "expenses in May")
```

4. Budget warning

```
def budget_warning(spent, budget):
    if spent > budget:
        return f"Over budget by {spent - budget:.2f}"
    elif spent >= 0.9 * budget:
        return f"Careful: only {budget - spent:.2f} left"
    else:
        return f"OK: {budget - spent:.2f} left"

print(budget_warning(112.5, 100))
print(budget_warning(95, 100))
print(budget_warning(60, 100))
```

5. Challenge: monthly report

```
expenses = [
    {"date": "2026-04-30", "category": "rent", "amount": 300.0, "note": "room"},
    {"date": "2026-05-01", "category": "food", "amount": 10.0, "note": "lunch"},
    {"date": "2026-05-02", "category": "fun", "amount": 18.0, "note": "concert"},
    {"date": "2026-05-09", "category": "food", "amount": 12.5, "note": "dinner"},
    {"date": "2026-05-20", "category": "travel", "amount": 5.0, "note": "train"},
]

def monthly_report(expenses, month):
    chosen = [e for e in expenses if e["date"].startswith(month)]
    total = 0
    totals = {}
    for e in chosen:
        total += e["amount"]
        totals[e["category"]] = totals.get(e["category"], 0) + e["amount"]
    top = None
    for category, amount in totals.items():
        if top is None or amount > totals[top]:
            top = category
    return {"count": len(chosen), "total": round(total, 2), "top_category": top}

print(monthly_report(expenses, "2026-05"))
```